Guía de componentes

Formularios, campos y modales del ERP GI — Next.js · react-hook-form · Zod · Tailwind · daisyUI

Contenido

1. useZodForm + FormLayout
2. Props comunes de todos los Fields
3. FieldText
4. FieldTextarea
5. FieldNumber
6. FieldSelect
7. FieldBoolean
8. FieldRadio
9. FieldChoice
10. FieldDate
11. FieldImage
12. FieldFile
13. toFormData — enviar archivos
14. Modales globales
15. Onchange — reaccionar a cambios

> **Cómo leer esta guía.** Cada componente trae una tabla con sus props y dos ejemplos. En los ejemplos, `FormInput` y `FormValues` son `z.input` y `z.output` del esquema del formulario.

Archivos: `src/components/ui/form/` (FormLayout y Fields) · `src/components/ui/modal/` · `src/hooks/useZodForm.ts` · `src/lib/toFormData.ts`

Versión del 27 de octubre de 2026

1. useZodForm + FormLayout

Todo formulario se arma con **useZodForm** (useForm + validación Zod) y **FormLayout**, que provee el contexto a los Fields, maneja el envío y avisa si el usuario intenta salir con cambios sin guardar (estilo Odoo).

Prop	Tipo	Descripción
`methods`	`UseFormReturn`	Resultado de `useZodForm(schema, opciones)`.
`onSubmit`	`(values) => void | Promise`	Recibe los valores ya validados y transformados por el esquema. Si lanza error, el form sigue "con cambios".
`onInvalid`	`(errors) => void`	Opcional. Se llama cuando la validación falla.
`guardUnsaved`	`boolean`	Aviso de cambios sin guardar al salir. **true** por defecto.
`className / …`	`props de <form>`	Cualquier prop nativa del form. Por defecto `flex flex-col gap-2`.

> **Al guardar con éxito** el formulario vuelve a quedar "sin cambios". El diálogo de salida ofrece *Seguir editando*, *Descartar cambios* y *Guardar y salir*. Cubre enlaces, botón atrás/adelante, recarga y cierre de pestaña; no bloquea `router.push()` llamado desde código.

Ejemplo 1 Alta de cliente

Esquema Zod, tipos inferidos y envío con carga y mensaje de éxito.

```tsx
"use client";
import { z } from "zod";
import useZodForm from "@/hooks/useZodForm";
import FormLayout from "@/components/ui/form/FormLayout";
import FieldText from "@/components/ui/form/FieldText";
import { withLoading, modalSuccess, modalError } from "@/components/ui/modal/modal";

const schema = z.object({
  name: z.string().min(1, "El nombre es obligatorio"),
  email: z.email("Ingresa un correo válido"),
});
type FormInput = z.input<typeof schema>;
type FormValues = z.output<typeof schema>;

export default function ClienteForm() {
  const methods = useZodForm(schema, { defaultValues: { name: "", email: "" } });

  const onSubmit = async (values: FormValues) => {
    try {
      await withLoading(api.createCliente(values), "Guardando cliente…");
      await modalSuccess("Cliente creado.");
    } catch (error) {
      modalError(error);
      throw error; // el form sigue marcado con cambios sin guardar
    }
```

```
  };

  return (
    <FormLayout methods={methods} onSubmit={onSubmit}>
      <FieldText<FormInput> name="name" label="Nombre" />
      <FieldText<FormInput> name="email" type="email" label="Correo" />
      <button type="submit" className="btn btn-primary">Guardar</button>
    </FormLayout>
  );
}
```

Ejemplo 2 Edición con datos de la API

Los `defaultValues` son el punto "sin cambios". Aquí se desactiva el aviso de salida y se usa `onInvalid`.

```
export default function EditarCliente({ cliente }: { cliente: Cliente }) {
  const methods = useZodForm(schema, {
    defaultValues: { name: cliente.name, email: cliente.email },
  });

  return (
    <FormLayout
      methods={methods}
      onSubmit={(values) => api.updateCliente(cliente.id, values)}
      onInvalid={() => modalWarning("Revisa los campos marcados en rojo.")}
      guardUnsaved={false}   // p. ej. en un filtro de búsqueda
      className="grid grid-cols-2 gap-4"
    >
      <FieldText<FormInput> name="name" label="Nombre" direction="row" />
      <FieldText<FormInput> name="email" label="Correo" direction="row" />
      <button type="submit" className="btn btn-primary col-span-2">Actualizar</button>
    </FormLayout>
  );
}
```

2. Props comunes de todos los Fields

Todos los Fields comparten estas props, además de las propias de cada uno. El valor, el error y el estado los toman del `FormLayout`, así que no se pasa `value`; `onChange` es opcional y solo sirve para reaccionar al cambio (el valor se guarda solo).

Prop	Tipo	Descripción
`name`	`FieldPath<T>`	Nombre del campo en el form (obligatorio). Con el genérico `<FormInput>` TypeScript lo autocompleta.
`label`	`string`	Etiqueta del campo.
`hint`	`string`	Texto de ayuda bajo el control; se reemplaza por el error cuando lo hay.
`direction`	`"col" | "row"`	`col` (defecto): label arriba. `row`: label a la izquierda si el campo mide ≥ 24rem.
`onChange`	`(value, form) => void`	Se ejecuta cuando el **usuario** cambia el valor (como `@api.onchange`). Ver sección 15.
`disabled`	`boolean`	Deshabilita el campo.
`rules`	`RegisterOptions`	Reglas de RHF. Solo para forms sin esquema Zod.

Ejemplo 1 Layout vertical (por defecto)

Label arriba, error o hint abajo.

```
<FieldText<FormInput> name="phone" label="Teléfono" hint="10 dígitos, sin espacios" />
```

Ejemplo 2 Layout horizontal

Label a la izquierda (10rem) y control a la derecha. En contenedores angostos vuelve a vertical solo.

```
<FieldText<FormInput>   name="rfc"   label="RFC"    direction="row" />
<FieldNumber<FormInput> name="limit" label="Límite" direction="row" widget="monetary" />
<FieldBoolean<FormInput> name="active" label="Activo" direction="row" variant="toggle" />
```

3. FieldText

Input de texto de daisyUI. Acepta cualquier prop nativa de `<input>` (`type`, `placeholder`, `maxLength`...).

Prop	Tipo	Descripción
`type`	`string`	`text` por defecto; también `email`, `password`, `tel`, etc.

Ejemplo 1 Correo y contraseña

```
// esquema: email: z.email("Correo inválido"), password: z.string().min(8, "Mínimo 8 caracteres")
<FieldText<FormInput> name="email" type="email" label="Correo" autoComplete="email" />
<FieldText<FormInput> name="password" type="password" label="Contraseña" />
```

Ejemplo 2 RFC en mayúsculas

La transformación vive en el esquema; el campo solo captura.

```
// esquema:
rfc: z.string().trim().toUpperCase().regex(/^[A-ZÑ&]{3,4}\d{6}[A-Z\d]{3}$/, "RFC inválido"),

<FieldText<FormInput> name="rfc" label="RFC" maxLength={13} placeholder="XAXX010101000" />
```

4. FieldTextarea

Textarea simple o, con `widget="html"`, un editor enriquecido (Tiptap) con títulos, listas, tablas, alertas de daisyUI, imágenes y atajos tipo Markdown (`##`, `**negrita**`, `-`, `>`). El editor solo se descarga donde se usa.

Prop	Tipo	Descripción
`widget`	`"text" | "html"`	Tipo de editor. `text` por defecto.
`rows`	`number`	Solo text. 3 por defecto.
`maxLength`	`number`	Solo text. Muestra el contador `120/500`.
`onUploadImage`	`(file) => Promise<string>`	Solo html. Sube la imagen y devuelve su URL; sin esto se inserta en base64.

> Al **mostrar** ese HTML en otra pantalla, límpialo antes (DOMPurify o en el backend) y usa la clase `rich-text prose` para que se vea igual que en el editor.

Ejemplo 1 Notas con límite

```
// esquema: notes: z.string().max(500, "Máximo 500 caracteres"),
<FieldTextarea<FormInput> name="notes" label="Notas" rows={4} maxLength={500} />
```

Ejemplo 2 Descripción en HTML con subida de imágenes

El valor es un string HTML (vacío = `""`).

```
// esquema: description: z.string().min(1, "Escribe la descripción"),
<FieldTextarea<FormInput>
  name="description"
  label="Descripción del producto"
  widget="html"
  onUploadImage={async (file) => {
    const res = await fetch("/api/uploads", { method: "POST", body: toFormData({ file }) });
    return (await res.json()).url;
  }}
/>
```

5. FieldNumber

Guarda siempre un `number` (nunca string). Al escribir muestra el número sin formato; al salir lo formatea (`$1,000.00`). Limita decimales **sin redondear**, alinea a la derecha y arranca en 0.

Prop	Tipo	Descripción
`widget`	`"float" | "integer" | "monetary"`	`float` por defecto.
`decimals`	`number`	Obligatorio con `float`. No se permite con los otros widgets.
`currency`	`string`	Solo monetary. `MXN` por defecto.
`locale`	`string`	`es-MX` por defecto.

Ejemplo 1 Precio y cantidad

```
// esquema:
price: z.number().positive("El precio debe ser mayor a 0"),
qty: z.number().int().min(1, "Mínimo 1 pieza"),

<FieldNumber<FormInput> name="price" label="Precio" widget="monetary" />
<FieldNumber<FormInput> name="qty"   label="Cantidad" widget="integer" />
```

Ejemplo 2 Peso con 3 decimales y moneda extranjera

```
<FieldNumber<FormInput> name="weight" label="Peso (kg)" decimals={3} />          // 1,234.500
<FieldNumber<FormInput> name="costUsd" label="Costo" widget="monetary" currency="USD" />
```

6. FieldSelect

Select nativo de daisyUI. Guarda el valor original de la opción (respeta `number`) o `null` sin selección. El tipo `SelectOption` se comparte con FieldRadio y FieldChoice.

Prop	Tipo	Descripción
options	SelectOption[]	{ value: string | number; label: string; disabled?: boolean }
placeholder	string	"Selecciona una opción" por defecto.

Ejemplo 1 Opciones fijas

```
// esquema: unit: z.enum(["pz", "kg", "lt"], { error: "Selecciona la unidad" }),
<FieldSelect<FormInput>
  name="unit"
  label="Unidad"
  options={[
    { value: "pz", label: "Pieza" },
    { value: "kg", label: "Kilogramo" },
    { value: "lt", label: "Litro" },
  ]}
/>
```

Ejemplo 2 Catálogo desde la API

Se mapean los registros al formato de opciones.

```
// esquema: warehouseId: z.number({ error: "Selecciona un almacén" }),
const options: SelectOption[] = almacenes.map((a) => ({
  value: a.id,
  label: a.nombre,
  disabled: !a.activo,
}));

<FieldSelect<FormInput> name="warehouseId" label="Almacén" options={options} />
```

7. FieldBoolean

Checkbox o toggle que guarda `true` / `false`.

Prop	Tipo	Descripción
`variant`	`"checkbox" | "toggle"`	checkbox por defecto.
`label`	`string`	Obligatorio. En `row` va en la columna del label.

Ejemplo 1 Aceptar términos

```
// esquema: terms: z.literal(true, { error: "Debes aceptar los términos" }),
<FieldBoolean<FormInput> name="terms" label="Acepto los términos y condiciones" />
```

Ejemplo 2 Toggle en layout horizontal

```
<FieldBoolean<FormInput> name="active" label="Activo" variant="toggle" direction="row" />
```

8. FieldRadio

Una opción de varias. Variante de círculos clásicos o grupo de botones (segmented).

Prop	Tipo	Descripción
`options`	`SelectOption[]`	Mismas opciones que FieldSelect.
`variant`	`"radio" | "buttons"`	`radio` por defecto.
`orientation`	`"vertical" | "horizontal"`	Acomodo de las opciones. `vertical` por defecto.

Ejemplo 1 Tipo de pago

```
// esquema: paymentType: z.enum(["cash", "credit"], { error: "Selecciona el tipo de pago" }),
<FieldRadio<FormInput>
  name="paymentType"
  label="Tipo de pago"
  orientation="horizontal"
  options={[{ value: "cash", label: "Contado" }, { value: "credit", label: "Crédito" }]}
/>
```

Ejemplo 2 Prioridad como botones

```
// esquema: priority: z.number({ error: "Selecciona la prioridad" }),
<FieldRadio<FormInput>
  name="priority"
  label="Prioridad"
  variant="buttons"
  orientation="horizontal"
  options={[{ value: 1, label: "Baja" }, { value: 2, label: "Media" }, { value: 3, label: "Alta" }]}
/>
```

9. FieldChoice

Varias opciones a la vez. Guarda un arreglo con los valores en el orden de la lista (no en el de los clics). Arranca en `[]`.

Prop	Tipo	Descripción
`options`	`SelectOption[]`	Mismas opciones que FieldSelect.
`variant`	`"checkbox" | "buttons"`	`checkbox` por defecto.
`orientation`	`"vertical" | "horizontal"`	`vertical` por defecto.

Ejemplo 1 Módulos con acceso

```
// esquema: modules: z.array(z.string()).min(1, "Selecciona al menos un módulo"),
<FieldChoice<FormInput>
  name="modules"
  label="Módulos"
  options={[
    { value: "sales", label: "Ventas" },
    { value: "purchases", label: "Compras" },
    { value: "inventory", label: "Inventario" },
  ]}
/>
```

Ejemplo 2 Días de entrega como botones

```
// esquema: days: z.array(z.number()).min(1, "Selecciona al menos un día"),
<FieldChoice<FormInput>
  name="days"
  label="Días de entrega"
  variant="buttons"
  orientation="horizontal"
  options={["Lun", "Mar", "Mié", "Jue", "Vie"].map((d, i) => ({ value: i + 1, label: d }))}
/>
```

10. FieldDate

Calendario de daisyUI (react-day-picker) en español. Guarda strings ISO, no objetos `Date`, para evitar desfases de zona horaria.

Prop	Tipo	Descripción
`mode`	`"single" | "range"`	`single` → `"2026-10-27"`. `range` → `{ from, to }`.
`withTime`	`boolean`	Solo single. Agrega hora → `"2026-10-27T14:30"`.
`min / max`	`"YYYY-MM-DD"`	Deshabilita los días fuera del rango.
`numberOfMonths`	`number`	Meses visibles. 1 por defecto.
`placeholder`	`string`	Texto cuando está vacío.

Ejemplo 1 Vencimiento y entrega con hora

```
// esquema:
dueDate: z.iso.date({ error: "La fecha es obligatoria" }),
deliveredAt: z.iso.datetime({ local: true }).nullable(),

<FieldDate<FormInput> name="dueDate" label="Vencimiento" min="2026-01-01" />
<FieldDate<FormInput> name="deliveredAt" label="Entregado" withTime />
```

Ejemplo 2 Periodo de un reporte

Primer clic = inicio, segundo = fin; si van al revés se ordenan solos.

```
// esquema:
period: z.object(
  { from: z.iso.date(), to: z.iso.date({ error: "Selecciona la fecha final" }) },
  { error: "Selecciona un periodo" },
),

<FieldDate<FormInput> name="period" label="Periodo" mode="range" numberOfMonths={2} />
```

11. FieldImage

Recuadro para elegir o arrastrar una imagen, con vista previa y botón para quitarla. Guarda un `File` nuevo, un `string` (URL actual al editar) o `null`.

Prop	Tipo	Descripción
shape	`"square" | "circle"`	square por defecto.
size	string	Clases de tamaño. `size-40` por defecto.
accept	string	png, jpeg, webp por defecto.

Ejemplo 1 Foto de perfil

```
// esquema:
avatar: z.union([
  z.file().max(2 * 1024 * 1024, "Máximo 2 MB")
    .mime(["image/png", "image/jpeg", "image/webp"], "Solo PNG, JPG o WEBP"),
  z.string(),
]).nullable(),

<FieldImage<FormInput> name="avatar" label="Foto" shape="circle" />
```

Ejemplo 2 Editar la imagen de un producto

Se pasa la URL actual como default; si no la cambian, se envía la misma URL.

```
const methods = useZodForm(schema, { defaultValues: { photo: producto.photoUrl } });

<FieldImage<FormInput> name="photo" label="Imagen" size="size-56" />
```

12. FieldFile

Zona para elegir o arrastrar un archivo. Muestra una tarjeta con ícono, nombre y tamaño; los PDF e imágenes se pueden ver en un modal (visor del navegador), los demás se descargan.

Prop	Tipo	Descripción
`accept`	`string`	PDF, imágenes, `.xlsx`, `.xls` y `.csv` por defecto.

Ejemplo 1 Contrato en PDF obligatorio

```
// esquema:
contract: z.union([
  z.file().max(10 * 1024 * 1024, "Máximo 10 MB").mime(["application/pdf"], "Solo PDF"),
  z.string(),
]).nullable().refine((v) => v !== null, "Adjunta el contrato"),

<FieldFile<FormInput> name="contract" label="Contrato" accept="application/pdf" />
```

Ejemplo 2 Importar un Excel

```
<FieldFile<FormInput>
  name="layout"
  label="Layout de precios"
  accept=".xlsx,.xls,.csv"
  hint="Descarga la plantilla desde Configuración > Precios"
/>
```

13. toFormData — enviar archivos

Convierte los valores del form a `FormData` (multipart). Archivos tal cual; arreglos simples como una entrada por elemento; objetos como JSON; `null` se omite. No pongas `Content-Type`: el navegador agrega el boundary.

Ejemplo 1 Crear con imagen y contrato

```
import { toFormData } from "@/lib/toFormData";

const onSubmit = async (values: FormValues) => {
  await fetch("/api/clientes", { method: "POST", body: toFormData(values) });
};
```

Ejemplo 2 Editar con campos extra

```
await fetch(`/api/clientes/${id}`, {
  method: "PUT",
  body: toFormData({ ...values, updatedBy: user.id }),
});
// period={"from":"2026-10-12","to":"2026-10-16"}  → el backend lo lee como JSON
```

14. Modales globales

Funciones globales que se pueden llamar desde cualquier parte, incluso fuera de React (helpers de fetch, interceptores). `<ModalProvider />` ya está montado en el layout raíz. Si se piden varios a la vez, se muestran en cola. También existe `useModal()`, que devuelve las mismas funciones.

Prop	Tipo	Descripción
`modalAlert / modalWarning`	`(message, options?) => Promise<void>`	Aviso informativo o de advertencia.
`modalSuccess`	`(message, options?) => Promise<void>`	Éxito con palomita animada y título "¡Listo!".
`modalError`	`(error, options?) => Promise<void>`	Acepta string, `Error` o lo capturado en un catch.
`modalConfirm`	`(message, options?) => Promise<boolean>`	`true` si confirma; `false` si cancela o pulsa Escape.
`alertLoading`	`(message?) => () => void`	Pantalla de carga con puntos; devuelve la función para cerrarla.
`withLoading`	`(promise, message?) => Promise<T>`	Muestra la carga mientras corre la promesa y la cierra siempre.
`options`	`{ title, confirmText, cancelText, danger }`	`danger` pone el botón principal en rojo (confirm).

Ejemplo 1 Avisos y errores

```
import { modalAlert, modalSuccess, modalError } from "@/components/ui/modal/modal";

modalAlert("Tu sesión expira en 5 minutos.");
await modalSuccess("Pedido enviado al almacén.", { title: "¡Enviado!" });

try {
  await api.timbrarFactura(id);
} catch (error) {
  modalError(error, { title: "No se pudo timbrar" });   // muestra error.message
}
```

Ejemplo 2 Confirmar, cargar y avisar

Flujo típico de una acción destructiva.

```
import { modalConfirm, withLoading, modalSuccess, alertLoading } from "@/components/ui/modal/modal";

const ok = await modalConfirm("¿Eliminar la factura F-0012? Esta acción no se puede deshacer.", {
  title: "Eliminar factura",
  confirmText: "Eliminar",
  danger: true,
});
if (!ok) return;
```

```
await withLoading(api.deleteFactura(id), "Eliminando…");
await modalSuccess("Factura eliminada.");

// Forma manual de la carga:
const close = alertLoading("Guardando nueva contraseña");
try { await api.changePassword(data); } finally { close(); }
```

15. Onchange — reaccionar a cambios

El equivalente a `@api.onchange` de Odoo. Hay dos formas y ambas se disparan **solo cuando el usuario cambia el valor**: los cambios hechos con `form.setValue()` (incluidos los del propio onchange) no las vuelven a disparar, así que un onchange puede actualizar otros campos sin entrar en bucle.

Prop	Tipo	Descripción
`onChange (prop)`	`(value, form) => void`	En cualquier Field. `value` trae el tipo del campo (number, boolean, File…); `form` es el form completo (`setValue`, `getValues`, `resetField`…). Para reacciones de un solo campo.
`useOnChange`	`(methods, campos[], handler) => void`	Hook a nivel form: `handler(values, { name, form })` se ejecuta cuando cambia cualquiera de los campos indicados, con todos los valores actuales. Ideal para campos calculados que dependen de varios.

> En campos de texto y números el onchange se dispara **con cada tecla**. Si llama a una API (p. ej. buscar un RFC), espera a que el usuario termine: usa un debounce o haz la consulta en el `onBlur`. Para cálculos locales no hace falta.

Ejemplo 1 onChange en un Field: cargar condiciones del cliente

Al elegir el cliente se consultan sus datos y se llenan otros campos. Puede ser `async`; combínalo con `withLoading` si la consulta tarda.

```
<FieldSelect<FormInput>
  name="customerId"
  label="Cliente"
  options={clientes}
  onChange={async (customerId, form) => {
    if (customerId == null) return;
    const cliente = await withLoading(api.getCliente(customerId), "Cargando cliente…");
    form.setValue("paymentTermDays", cliente.diasCredito, { shouldDirty: true });
    form.setValue("priceListId", cliente.listaPrecios, { shouldDirty: true });
  }}
/>

// Otro uso común: limpiar un campo dependiente
<FieldBoolean<FormInput>
  name="requiresInvoice"
  label="Requiere factura"
  onChange={(checked, form) => { if (!checked) form.setValue("rfc", ""); }}
/>
```

Ejemplo 2 useOnChange: subtotal, IVA y total

Como `@api.onchange('qty', 'price', 'discount')`. Los campos calculados se pueden mostrar con `disabled` para que el usuario no los edite.

```
import useOnChange from "@/hooks/useOnChange";

const methods = useZodForm(schema, { defaultValues });

useOnChange(methods, ["qty", "price", "discount"], (values, { form }) => {
  // FieldNumber vale null mientras el usuario borra el campo
  const { qty = 0, price = 0, discount = 0 } = values;
  const subtotal = (qty ?? 0) * (price ?? 0) * (1 - (discount ?? 0) / 100);
  form.setValue("subtotal", subtotal);
  form.setValue("tax", subtotal * 0.16);
  form.setValue("total", subtotal * 1.16);
});

<FieldNumber<FormInput> name="qty"      label="Cantidad"  widget="integer" />
<FieldNumber<FormInput> name="price"    label="Precio"    widget="monetary" />
<FieldNumber<FormInput> name="discount" label="Desc. %"   decimals={2} />
<FieldNumber<FormInput> name="total"    label="Total"     widget="monetary" disabled />
```